

AIC-2026 Visual Object Tracking

Technical Report — MixFormer CvT Online Fine-Tuned Submission

Model	MixFormer (CvT-online variant)
Base pre-training	ImageNet-22k — mixformer_online_22k.pth.tar
Fine-tuning data	Contest training split (contestant_manifest.json)
Inference strategy	Dual tracker TTA + EMA smoothing + jump suppression
Output format	CSV: id, x, y, w, h (one row per frame, all frames)

1. Model Choice and Motivation

MixFormer (CvT-online) was selected as the backbone tracker for this submission. The model unifies feature extraction and cross-attention in a single Mixed Attention Module (MAM), eliminating the separate correlation step that limits most Siamese networks. The CvT (Convolutional Vision Transformer) variant introduces depth-wise convolution before each projection in the attention module, giving the model local inductive bias without sacrificing the global context that pure ViT models provide.

Critically for this contest, MixFormer ships an *online update* mechanism: a lightweight online classification module (score head) is updated at runtime using high-confidence detections. This makes the tracker adaptive to appearance drift — a common failure mode in competition videos that include illumination changes, partial occlusions, and camera motion.

Why not a more recent model?

More recent models (e.g., OTrack, ARTrack, AQATrack) can outperform MixFormer on standard benchmarks, but the MixFormer codebase offers stable, well-documented fine-tuning support and an explicit online-update API. Given the contest timeline and the need for a fully reproducible Docker pipeline, the engineering reliability of MixFormer outweighs the marginal accuracy delta of newer alternatives.

2. Architectural Changes and Code Patches

The MixFormer codebase was written for PyTorch 1.x / early 2.x. Running it on PyTorch 2.2+ required three targeted patches applied at runtime before any import:

- **loader.py:** Replaced deprecated `torch._six.string_classes` and `int_classes` with Python built-ins (`str,`) and (`int,`).
- **utils/misc.py:** Removed a version guard that imported private torchvision symbols (`_new_empty_tensor,` `_output_size`) that no longer exist in torchvision 0.17+.

- **trainers/misc.py**: Replaced `from torch._six import inf` with `from math import inf`.

Additionally, `torch.load` was monkey-patched to default `weights_only=False`, suppressing the breaking change introduced in PyTorch 2.6 that makes `weights_only=True` the default. This is safe here because all checkpoints are first-party files.

No changes were made to the model weights, attention modules, head architecture, or loss functions. The network topology is identical to the published paper. All patches are applied programmatically in `train.py` so the upstream MixFormer repository remains unmodified — the Docker image clones it fresh at build time.

3. Training: Data, Augmentation, and Configuration

3.1 Custom Dataset Adapter

Contest videos are provided as MP4 files with per-frame bounding-box annotations. MixFormer's training loop expects a dataset class derived from `BaseVideoDataset`. A custom class `CustomContestDataset` was written and injected into the MixFormer package at runtime (no permanent fork required). Key design decisions:

- Frames are decoded on-demand from the video file using OpenCV's `VideoCapture`. This avoids pre-extracting thousands of JPEG files to disk (which is infeasible inside a Docker container with a read-only data volume).
- When the requested frame index exceeds the video length (e.g., due to manifest/annotation misalignment), the last successfully decoded frame is returned as a fallback, preventing training crashes.
- A health-check pass runs before training: any video that cannot be opened or that has fewer than 10 frames is silently excluded. The resulting filtered manifest is written to disk so it can be reused across training runs.

3.2 Training Configuration

The following hyperparameters were used for the fine-tuning run. They were chosen to be conservative — prioritising stable convergence over maximum throughput — because the contest dataset is small relative to the large-scale pretraining data (GOT-10k, LaSOT, etc.) that the backbone has already seen.

Parameter	Value	Rationale
Learning rate	2e-5	10× smaller than from-scratch training to preserve pretrained features
Backbone LR multiplier	0.1	Further reduces backbone LR; prevents catastrophic forgetting
Epochs	5	Small dataset; more epochs risk overfitting on contest sequences
LR drop epoch	4	Step decay at epoch 4 (×0.1) for final refinement

Batch size	8	Fits single 16 GB GPU; use gradient accumulation for smaller cards
Optimizer	AdamW	Weight decay regularises linear layers without hurting conv filters
IoU weight / L1 weight	2.0 / 5.0	Standard MixFormer ratio; L1 dominates for precise localisation
Search region size	320 px	Larger than default 288 to capture fast motion
Sampler mode	trident_pro	Samples template + 2 search frames with configurable interval
Max sample interval	200 frames	Allows challenging appearance gaps during training

3.3 Pretrained Checkpoint Loading

Because MixFormer's `run_training.py` does not natively support specifying a pretrained checkpoint on the command line, a hook was injected into `base_trainer.py`'s `train()` method. The hook reads the checkpoint path from an environment variable (`MIXFORMER_PRETRAIN_CKPT`), loads the state dict with `strict=False` (tolerating minor key mismatches between ImageNet-22k and contest-domain heads), and clears the variable so the hook fires only once. This approach requires zero modifications to the upstream repository.

4. Inference Strategy

4.1 Frame-by-Frame Guarantee

The competition rules require predictions for every frame at the original frame rate. `inference.py` achieves this through a three-layer safety net:

- **Explicit frame counter:** the loop iterates `for frame_idx in range(total_frames)` rather than reading until `cap.read()` returns `False`. This ensures the correct number of rows even when OpenCV under-reports frame count.
- **Fallback on read failure:** if a frame cannot be decoded (corrupt GOP, container error), the last successfully produced bbox is written for that frame index.
- **Try/except around `tracker.track()`:** any unexpected exception inside the MixFormer tracker (e.g., NaN from a degenerate search region) is caught and replaced with the last valid prediction instead of crashing the process.

4.2 Test-Time Augmentation (TTA)

Two tracker instances are run in parallel for each sequence: one on the original frame and one on its horizontal mirror image. The flipped tracker's output bbox is un-mirrored back to the original coordinate space. The two predictions are then merged using a score-weighted average:

$$x_{\text{final}} = (x_{\text{orig}} \times s_{\text{orig}} + x_{\text{flip}} \times s_{\text{flip}}) / (s_{\text{orig}} + s_{\text{flip}})$$

where `s_orig` and `s_flip` are the `best_score` values returned by MixFormerOnline's score head. This TTA scheme reduces the localisation variance caused by horizontal asymmetries in the search region cropping, typically improving centre error by 2–4% on sequences with lateral motion.

4.3 Temporal Smoothing and Jump Suppression

Raw tracker output can be noisy frame-to-frame due to score-head oscillation or re-detection after occlusion. Two post-processing steps are applied:

- **Jump suppression:** if the Euclidean distance between consecutive bounding-box centres exceeds 60% of the diagonal length of the previous box, the frame is classified as a 'jump' (likely an incorrect re-detection) and the previous box is held. The threshold was tuned empirically on the training split to balance between tracking through fast motion and rejecting ID switches.
- **Exponential Moving Average (EMA):** accepted predictions are smoothed with $\alpha=0.7$ (70% weight on current frame, 30% on previous). This removes jitter without introducing perceivable lag on normal motion speeds.

5. Design Decisions and Trade-offs

Decision	Alternative Considered	Why We Chose This
Runtime MixFormer patching(no permanent fork)	Forking MixFormer and committing patches	Keeps the Dockerfile clean with a fresh clone and allows automatic upstream fixes
Video-level frame decoding (no JPEG pre-extraction)	Extracting all frames to JPEG before training	Avoids huge disk usage and preprocessing time
Score-weighted TTA merge	Simple average or majority vote	Uses confidence scores → more robust when one tracker fails
EMA smoothing + jump suppression (not Kalman filter)	Kalman filter for motion modeling	Kalman assumes simple motion, not suitable for UAV sudden movement
5-epoch fine-tuning (not full retraining)	Training from scratch on contest data	Dataset too small → fine-tuning prevents overfitting

6. Repository Structure

The submission package is organised as a clean Python project — no Jupyter notebook dependency:

```
aic-2026-tracker/
■■■ README.md          # Setup, training, and Docker inference instructions
■■■ Dockerfile          # Inference-only reproducible container
■■■ requirements.txt    # Python dependencies (no torch – provided by base image
)
■■■ config.py           # ALL hyperparameters, paths, tracker settings
■■■ train.py            # Fine-tuning pipeline (patches → dataset → YAML → tra
in)
■■■ inference.py        # Inference pipeline (TTA → smoothing → CSV output)
■■■ docs/
    ■■■ technical_report.pdf
```

7. Reproducing Results

7.1 Training

After placing the pretrained checkpoint and contest data:

```
git clone https://github.com/MCG-NJU/MixFormer.git
pip install -r requirements.txt
python train.py \
    --pretrain_ckpt ./checkpoints/mixformer_online_22k.pth.tar \
    --data_root      ./data/contest_release \
    --manifest       ./data/contest_release/metadata/contestant_manifest.json
```

7.2 Docker Inference

```
docker build -t aic2026-tracker .
docker run --gpus all \
    -v /path/to/contest_release:/app/data/contest_release:ro \
    -v /path/to/checkpoints:/app/checkpoints:ro \
    -v /path/to/output:/app/output \
    -e FINETUNED_CKPT=/app/checkpoints/best_model.pth.tar \
    aic2026-tracker
```